# Agrifood Systems Value Added - Step 4a, Eora data extraction

October 17, 2024

## 1 Eora Data Extraction

### 1.1 Script Background

- This script builds upon the **ASTAR-Labor project** (led by Dr. Chris Barrett at Cornell University) to compute transactions between industries along the agri-food value chain.
- A balancing methodology was developed through the ASTAR-Labor project (Yi et al, 2024) to satisfy Walras's Law, and the balanced tables are generated based on **EORA 26** raw tables.
- More information about the EORA data is available at: https://worldmrio.com/eora26/

### 1.2 Script Functionality

This script is designed to:

- Develop the gross output by country-sector.
- Calculate domestic transactions between sectors.
- Aggregate imports and exports by sector.
- Develop the margin estimations.

#### 1.2.1 Importing Libraries and User-Defined Functions

Below is the code to import the necessary libraries and define custom functions for the analysis. These libraries are publicly available.

In addition, we include user-defined functions specifically developed for this project to assist with data processing.

[4]:
```
import os
import pandas as pd
import time
import numpy as np
import warnings
warnings.filterwarnings("ignore")
import sys
from scipy.stats.mstats import winsorize
import seaborn as sns
import matplotlib.pyplot as plt
import statsmodels.api as sm
```

```
import plotly.graph_objects as go

def fn_FdAbb(df_temp,colName):
    conditions = [(df_temp[colName] =='Household final consumption P.3h'),
                  (df_temp[colName] == 'Non-profit institutions serving␣
 ↪households P.3n'),
                  (df_temp[colName] == 'Government final consumption P.3g'),
                  (df_temp[colName] == 'Gross fixed capital formation P.51'),
                  (df_temp[colName] == 'Changes in inventories P.52'),
                  (df_temp[colName] == 'Acquisitions less disposals of␣
 ↪valuables P.53')]
    results = ["XH","XNPISH","XG","XK01","XK02","XK03"]
    colNew = colName+"Abb"
    df_temp[colNew] = np.select(conditions,results)
    return df_temp

def A01T02(df):
    # df = X_filtered_iA1T4_exp_agg
    df = df.rename(columns = lambda x: x.replace('_A01','_A01T02').
 ↪replace('_A02','_A01T02'))
    df = df.rename(index=lambda x: x.replace('A01', 'A01T02').replace('A02',␣
 ↪'A01T02'))
    df_a0102 = df.groupby(df.columns,axis=1).sum()
    df_a0102.sort_index(axis=1,inplace=True)
    df_a0102 = df_a0102.groupby(df_a0102.index).sum()
    return df_a0102


def sum_col(df_i, df_fs_cord):
    df_fs_row = df_fs_cord.T
    result = pd.DataFrame()
    # for col_i in df_i.columns:
    #     # only work with food service related columns:
    #     if col_i in df_fs_row.columns:
    #         result[col_i] = df_i[col_i] * df_fs_row[col_i].values[0]
    for col_i in df_i.columns:
        # only work with food service related columns
        if col_i in df_fs_row.columns:
            if df_fs_row[col_i].shape[0] > 0:  # Check if there are values in␣
 ↪the column
                result[col_i] = df_i[col_i] * df_fs_row[col_i].values[0]
            else:
                result[col_i] = df_i[col_i] * 1  # Filling with 1 when the␣
 ↪column is empty

    group_key = [col[:3] for col in result.columns.get_level_values(0)]
```

```
    df_i_sum = result.groupby(group_key, axis=1).sum()
    df_i_sum = df_i_sum.add_suffix("_A18")
    return df_i_sum


def get_comm_rows_cols(df_1_ori,df_2_ori):
    df_1 = df_1_ori.copy()
    df_2 = df_2_ori.copy()
    df_1.columns = df_1.columns.str[:3]
    df_2.columns = df_2.columns.str[:3]

    df_1.index = df_1.index.str[:3]
    df_2.index = df_2.index.str[:3]

    common_columns = df_1.columns.intersection(df_2.columns)
    df_1 = df_1[common_columns]
    df_2 = df_2[common_columns]
    common_index = df_1.index.intersection(df_2.index)
    df_1 = df_1.loc[common_index]
    df_2 = df_2.loc[common_index]
    return df_1,df_2

def Margin_win(df):
    df_ratio = df.rename_axis('ROW').reset_index()
    df_ratio = pd.melt(df_ratio, id_vars=['ROW'], var_name='COL',␣
 ↪value_name='Trans_ratio')

    q1 = df_ratio['Trans_ratio'].quantile(0.25)
    q3 = df_ratio['Trans_ratio'].quantile(0.75)
    iqr = q3 - q1
    lower_lim = q1 - 1.5 * iqr
    upper_lim = q3 + 1.5 * iqr
    # print('Outliers:',df_ratio[df_ratio['Trans_ratio'] > upper_lim])
    df_ratio['Trans_ratio'].clip(upper=upper_lim, inplace=True)
    return df_ratio
```

### 1.3 Set up the Local Directories

**Note:** - The folders are used for temporary data storage and processing. - The balanced tables imported in the loop below with the following command were processed by Yi et al. (2024). As the source data is proprietary, please consult EORA regarding your rights and permissions to use the raw data.

[ ]:
```
# "..." indicates omitted subfolder names in the following paths:
WorkingDir = r"D:\...\Code"
OutputDir = WorkingDir + "/Data"
OutputMargin = OutputDir + "/Margin"
```

```
DirVad = WorkingDir + "/Validation"
```

### 1.4 Set Pandas Display Options:

[6]:
```
pd.set_option('display.max_columns', 15)
pd.set_option('display.max_row', 15)
pd.set_option('display.width', None)
pd.set_option('display.max_colwidth', -1)
pd.set_option('display.float_format',lambda x:'%.3f' % x)
```

### 1.5 To develop aggregated exports:

- we first linearize the balanced table by using the following code in the last cell: "'python df_bal_yr3 = pd.melt(df_bal_yr3, id_vars=['ROW'], var_name='COL', value_name='Value')

[29]:
```
print(df_test)
```

```
         Unnamed: 0      ROW      COL   Value
0                 0  ABW_A01  ABW_A01   0.141
1                 1  ABW_A02  ABW_A01   0.590
2                 2  ABW_A03  ABW_A01   3.958
3                 3  ABW_A04  ABW_A01  74.832
4                 4  ABW_A05  ABW_A01   2.364
…               …       …        …      …
7699465     7699465  LSO_A03  FRA_A07   5.141
7699466     7699466  LSO_A04  FRA_A07  10.021
7699467     7699467  LSO_A05  FRA_A07   1.544
7699468     7699468  LSO_A06  FRA_A07   1.360
7699469     7699469  LSO_A07  FRA_A07  74.205

[7699470 rows x 4 columns]
```

- Similar to the process of imports, the code below identifies the domestic demand and exported demand.
- The domestic demand and the aggregated exports of the 'ABW_A01' sector is (Note: the last few rows with the prefix 'Exp' indicates the aggregated exports):

[31]:
```
print(df_test)
```

```
    Unnamed: 0      ROW      COLNew    Value
0            0  ABW_A01     ABW_A01    0.141
1            1  ABW_A01     ABW_A02    0.092
2            2  ABW_A01     ABW_A03    2.507
3            3  ABW_A01     ABW_A04  614.571
4            4  ABW_A01     ABW_A05    3.381
5            5  ABW_A01     ABW_A06  157.747
6            6  ABW_A01     ABW_A07   96.990
```

| | | | | |
|---|---|---|---|---|
| 7 | 7 | ABW_A01 | ABW_A08 | 0.188 |
| 8 | 8 | ABW_A01 | ABW_A09 | 0.167 |
| 9 | 9 | ABW_A01 | ABW_A10 | 0.108 |
| 10 | 10 | ABW_A01 | ABW_A11 | 1.059 |
| 11 | 11 | ABW_A01 | ABW_A12 | 0.121 |
| 12 | 12 | ABW_A01 | ABW_A13 | 0.960 |
| 13 | 13 | ABW_A01 | ABW_A14 | 75.081 |
| 14 | 14 | ABW_A01 | ABW_A15 | 0.310 |
| 15 | 15 | ABW_A01 | ABW_A16 | 4.684 |
| 16 | 16 | ABW_A01 | ABW_A17 | 4.703 |
| 17 | 17 | ABW_A01 | ABW_A18 | 331.753 |
| 18 | 18 | ABW_A01 | ABW_A19 | 1.516 |
| 19 | 19 | ABW_A01 | ABW_A20 | 0.429 |
| 20 | 20 | ABW_A01 | ABW_A21 | 123.641 |
| 21 | 21 | ABW_A01 | ABW_A22 | 10.179 |
| 22 | 22 | ABW_A01 | ABW_A23 | 31.431 |
| 23 | 23 | ABW_A01 | ABW_A24 | 0.306 |
| 24 | 24 | ABW_A01 | ABW_A25 | 0.137 |
| 25 | 25 | ABW_A01 | ABW_A26 | 0.143 |
| 26 | 26 | ABW_A01 | ABW_XG | 1.598 |
| 27 | 27 | ABW_A01 | ABW_XH | 674.178 |
| 28 | 28 | ABW_A01 | ABW_XK01 | 11.758 |
| 29 | 29 | ABW_A01 | ABW_XK02 | 71.063 |
| 30 | 30 | ABW_A01 | ABW_XK03 | 0.042 |
| 31 | 31 | ABW_A01 | ABW_XNPISH | 13.093 |
| 32 | 157192 | ABW_A01 | Exp_A01 | 186.625 |
| 33 | 157193 | ABW_A01 | Exp_A02 | 84.448 |
| 34 | 157194 | ABW_A01 | Exp_A03 | 212.768 |
| 35 | 157195 | ABW_A01 | Exp_A04 | 347.928 |
| 36 | 157196 | ABW_A01 | Exp_A05 | 264.787 |
| 37 | 157197 | ABW_A01 | Exp_A06 | 190.059 |
| 38 | 157198 | ABW_A01 | Exp_A07 | 361.478 |
| 39 | 157199 | ABW_A01 | Exp_A08 | 210.040 |

### 1.6 The loop:

[ ]:
```
for year_i in range(1995, 2022):
    start_time = time.time()
    df_bal_yr3 = pd.read_csv(DirVad + '/jhu_bal' + str(year_i) + ".csv",␣
 ↪sep='|')
    df_bal_yr3.set_index('ROW',inplace=True)
    # aggregate 'the rest of the world':
    df_bal_yr3['ZZZ'] = df_bal_yr3['ZZZ_ZLR1'] + df_bal_yr3['ZZZ_ZX1142']
    df_bal_yr3.loc['ZZZ'] = df_bal_yr3.loc[['ZZZ_ZLR1', 'ZZZ_ZL0008']].sum()
    df_bal_yr3 = df_bal_yr3.drop(columns=['ZZZ_ZLR1', 'ZZZ_ZX1142'])
    df_bal_yr3 = df_bal_yr3.drop(['ZZZ_ZLR1', 'ZZZ_ZL0008'])
    # gross output by country and sector:
```

```
  RowTot = df_bal_yr3.sum(axis=1)
  dfRowTot = pd.DataFrame(RowTot,columns=['GrossOutput'])
  dfRowTot.reset_index('ROW',inplace=True)
  # rows for leakage:
  NotT_row = ['LH01', 'LG01', 'LG02', 'LK01', 'LK02', 'LK03']
  #------------ collapse imports:---------
  df_output = pd.DataFrame()
  for col in df_bal_yr3.columns:
      # print(col)
      prefix = col.split('_')[0]
      df_i = df_bal_yr3[[col]].copy()
      # df_i.rename(columns={col:'Value'},inplace=True)
      df_i['Dest'] = col.split('_')[0]
      # Apply the function to create the 'Origin' column
      df_i['Origin'] =  df_i.apply(lambda row: row.name[:3] if "_" in str(row.
↪name) else ('ZZZ' if 'ZZZ' in row.name else row.name), axis=1)
      df_i['Origin_Sec'] = df_i.apply(lambda row: row.name[4:] if "_" in␣
↪str(row.name) else  ('ZZZ' if 'ZZZ' in row.name else row.name), axis=1)
      # identify domestic supply:
      df_dom = df_i[(df_i['Dest'] == df_i['Origin']) | df_i.index.astype(str).
↪isin(['LH01', 'LG01', 'LG02', 'LK01', 'LK02','LK03'])]
      # identify imports:
      df_imp = df_i[(df_i['Dest'] != df_i['Origin']) & ~df_i.index.
↪astype(str).isin(['LH01', 'LG01', 'LG02', 'LK01','LK02', 'LK03'])]
      df_impTot = df_imp.groupby('Origin_Sec')[col].sum().reset_index()

      df_dom['ROW'] = df_dom['Origin'].astype(str) + "_" +␣
↪df_dom['Origin_Sec'].astype(str)
      df_dom['Value'] = df_dom[col]
      df_dom['COL'] = col
      df_impTot['ROW'] = "Imp_"+df_impTot['Origin_Sec'].astype(str)
      df_impTot['Value'] = df_impTot[col]
      df_impTot['COL'] = col
      df_i_proc = pd.concat([df_dom[['ROW','COL','Value']],␣
↪df_impTot[['ROW','COL','Value']]])
      df_i_proc.reset_index(drop=True, inplace=True)
      df_output = df_output.append(df_i_proc)

  df_TransOutput = pd.merge(df_output,dfRowTot, on='ROW')
  df_TransOutput['Coef'] = df_TransOutput['Value']/
↪df_TransOutput['GrossOutput']

  df_TransOutput_col = pd.merge(df_output,dfRowTot,␣
↪left_on='COL',right_on='ROW' )
  df_TransOutput_col['Coef'] = df_TransOutput_col['Value']/
↪df_TransOutput_col['GrossOutput']
```

```
   df_TransOutput_col_row = pd.merge(df_TransOutput_col, dfRowTot,␣
↪left_on='ROW_x', right_on='ROW', how='left')
   df_TransOutput_col_row.rename(columns={'GrossOutput_y':
↪'GrossOutput_Row','GrossOutput_x':'GrossOutput_Col'},
                                 inplace=True)
   df_TransOutput_col_row['ROW'] = df_TransOutput_col_row['ROW_x']
   df_TransOutput_col_row['Coef_Row'] = df_TransOutput_col_row['Value']/
↪df_TransOutput_col_row['GrossOutput_Row']
   df_TransOutput_col_row['Coef_Col'] = df_TransOutput_col_row['Value']/
↪df_TransOutput_col_row['GrossOutput_Col']
   df_TransOutput_col_row['Year'] = year_i
   df_TransOutput_col_rowOut =␣
↪df_TransOutput_col_row[['Year','ROW','COL','Value','GrossOutput_Row',
                                                          ␣
↪'Coef_Row','GrossOutput_Col','Coef_Col']]
   df_TransOutput_col_rowOut.to_csv(OutputDir+'/Coeff_Both_'+str(year_i)+'.
↪csv',sep='|',index=False)
   print('Imported were processed')

   # ----------------------process for exports:----------------
   df_bal_yr3 = df_bal_yr3[~df_bal_yr3.index.isin(NotT_row)]
    # rewrite the export code:
   df_bal_yr3.reset_index(inplace=True)
   df_bal_yr3 = pd.melt(df_bal_yr3, id_vars=['ROW'], var_name='COL',␣
↪value_name='Value')
   df_bal_yr3['Domestic'] = np.where(
       df_bal_yr3['COL'].str[:3] == df_bal_yr3['ROW'].str[:3], 1, 0)
   mask = df_bal_yr3['COL'].str.contains('_')
   df_bal_yr3['COL_Sec'] = np.where(mask, df_bal_yr3['COL'].str.split('_').
↪str[1], 'ZZZ')

   domestic_agg = df_bal_yr3[df_bal_yr3['Domestic'] == 1].
↪groupby(['ROW','COL'])[
       'Value'].sum().reset_index()
   exp_agg = df_bal_yr3[df_bal_yr3['Domestic'] == 0].groupby(['ROW',␣
↪'COL_Sec'])[
       'Value'].sum().reset_index()
   result_df = pd.concat([domestic_agg, exp_agg], ignore_index=True)
   result_df['COLNew'] = np.where(~result_df['COL'].isnull(),result_df['COL'],␣
↪'Exp_'+result_df[
       'COL_Sec'])
   result_df = result_df[['ROW','COLNew','Value']]
   result_df.rename(columns={'COLNew':'COL'})
   result_df.to_csv(OutputDir +'/exp' + str(year_i) + '.csv',sep='|')
```

```
    end_time = time.time()
    elapsed_time = end_time - start_time
    print(f"{year_i} Time elapsed of each year (without processing the balanced␣
↪table):"
          f" {elapsed_time:.4f} "
          f"seconds")
```

# 2 Margins

This section estimates the margin industries' contributions to the global agri-food systems.

Margins represent the value of the retail, wholesale, and transportation services provided in delivering commodities from producers' establishments to purchasers (Implan)

Using the EORA dataset, we estimate these values for the margin industries (transportation and trade) by utilizing the transportation and trade margin tables. The dimension of each annual dataset of the transportation margin table is 14,839 by 14,839, representing the transportation services for the transactions between 14,839 industries across 189 countries.

### 2.0.1 Methodology

To estimate the margin markup, we follow these steps:

1. We first calculate the markup ratio by using food service industries' demand for transportation and trade services.
2. The margin markup is calculated as:

$$\frac{\text{Transportation Services Paid by Food Service Industries}}{\text{Commodities/Services Purchased by Food Service Industries at Basic Prices}}$$

3. The markup ratio is then applied to final consumers' demand for food (at basic prices) to estimate the corresponding margins for food.
4. To disaggregate trade margins between wholesale and retail industries, household demand for wholesale and retail services is used to allocate the total trade margin between these two sectors.

## 2.1 Prepare the column names of the EORA data

[ ]:
```
fileName = "/labels_FD.txt"
preName = "X"
df_i_header_file = OutputDir + fileName
df_FD_header = pd.read_csv(df_i_header_file, sep=r'\t',␣
↪engine='python',header=None )
df_FD_header.columns = [['Abb','Abb2','Category','Description']]
df_FD_header = fn_FdAbb(df_FD_header,'Description')
df_FD_header['ctry_col_abb'] = df_FD_header['Abb',].astype(str)␣
↪+"_"+df_FD_header['DescriptionAbb',]
df_FD_header.columns␣
↪=['Abb','Abb2','Category','Description','DescriptionAbb','Ctry_Col_Abb']
```

```
FullHead = OutputDir+"/FullEora/indices"
df_Head_t_full = pd.read_csv(FullHead+"/index_t.csv")
df_Head_t_full['Index'] = df_Head_t_full['Index'].astype(str).apply(lambda x:x.
 ↪zfill(5))
df_Head_t_full['Ctry_Index'] =␣
 ↪df_Head_t_full['CountryA3']+"_"+df_Head_t_full['Index'].astype(str)
df_Head_t_full = df_Head_t_full.rename(columns={' Entity':'Entity'})
df_Head_t_full.Ctry_Index = np.where(df_Head_t_full.
 ↪Ctry_Index=='ROW_14839','ZZZ_ROW', df_Head_t_full.Ctry_Index)

df_cord = pd.read_csv(OutputDir+"/conc_full2simplified.txt",header=0,sep="\t")
df_cord=df_cord.rename(columns={'Unnamed: 0':'Country', 'Unnamed: 1':
 ↪"CountryA3", "Unnamed: 2":"Entity","Unnamed: 3":"Sector"})

df_cord_full = pd.concat([df_Head_t_full, df_cord.iloc[:,4:]], axis=1)
df_cord_full['Ctry_Ent_Sec'] = df_cord_full['CountryA3'] +␣
 ↪'_'+df_cord_full['Entity'] +"_"+ df_cord_full['Sector']
df_cord_full.set_index(['Ctry_Index','Ctry_Ent_Sec'],inplace=True)
df_ctry = pd.read_csv(OutputDir + '/ctry.csv',sep = '|')
```

### 2.2 Develop the transportation margin:

[ ]:
```
## Transportation margin development#
for year_i in range(1993, 2022):
#     Import balanced EORA 26 tables:
    df_out = pd.read_csv(OutputDir+'/Export/Exp_Agg_A27'+str(year_i)+'.
 ↪csv',sep='|', index_col='ROW')
# Get the inter-industry, final demand, and the leakage matrices:
    X_col = [col for col in df_out.columns if col[4:5] == 'X' ]
    T_ = df_out.drop(X_col, axis=1)
    l_list = ['LH01', 'LG01', 'LG02', 'LK01', 'LK02', 'LK03']
    T_ = T_[~T_.index.isin(l_list)]
    L_ = df_out[df_out.index.isin(l_list)]
    L_ = L_.drop(X_col, axis=1)
    X_ = df_out[X_col]
    X_ = X_[~X_.index.isin(l_list)]

    Ttemp = T_.copy()
    Ttemp = Ttemp.reset_index()
    Ttemp = Ttemp.melt(id_vars=['ROW'], var_name='COL',value_name='Value')
    negative_rows = Ttemp[Ttemp['Value'] < 0].copy()
    negative_rows_mapping = negative_rows.rename(columns={'ROW': 'COL', 'COL':␣
 ↪'ROW'})
    merged_df = pd.merge(Ttemp, negative_rows_mapping[['ROW', 'COL', 'Value']],␣
 ↪on=['ROW', 'COL'], how='left', suffixes=('', '_to_add'))
```

```
    Ttemp['Value'] = merged_df.apply(lambda row: row['Value'] + (-1) *␣
 ↪row['Value_to_add'] if pd.notnull(row['Value_to_add']) else row['Value'],␣
 ↪axis=1)
    Ttemp.loc[negative_rows.index, 'Value'] = 0
    Ttemp = Ttemp.pivot(index='ROW', columns='COL', values='Value')
    Ttemp = Ttemp.rename_axis('ROW')
    T_ = Ttemp.copy()
#     Aggregate the agriculture and fishing industry:
    T25 = A01T02(T_)
    L25 = A01T02(L_)
    X25 = A01T02(X_)
# Import the transportation margin table:
    if year_i==2020:
        print(year_i)
        df_trans_marg = pd.read_csv(dir + "/
 ↪20200801_Mother_AllCountries_199_T-Results_2020_093_Markup003.
 ↪csv",header=None)
    elif year_i == 2016:
        df_trans_marg = pd.read_csv(dir + "/
 ↪20200623_Mother_AllCountries_199_T-Results_" + str(
            year_i) + "_090_Markup003.csv",header=None)
    elif year_i == 2017:
        df_trans_marg = pd.read_csv(dir + "/
 ↪20200624_Mother_AllCountries_199_T-Results_" + str(
            year_i) + "_090_Markup003.csv",header=None)
    elif year_i == 2018:
        df_trans_marg = pd.read_csv(dir + "/
 ↪20200628_Mother_AllCountries_199_T-Results_" + str(
            year_i) + "_090_Markup003.csv",header=None)
    elif year_i == 2019:
        df_trans_marg = pd.read_csv(dir + "/
 ↪20200801_Mother_AllCountries_199_T-Results_2019_093_Markup003.csv",
                                    header=None)
    elif year_i == 2021:
        df_trans_marg = pd.read_csv(dir +'/
 ↪20200802_Mother_AllCountries_199_T-Results_2021_093_Markup003.csv',
                                    header = None)
    elif 2011 < year_i <2016:
        print(year_i)
        df_trans_marg = pd.read_csv(dir + "/
 ↪21160000_annetest_AllCountries_199_T-Results_" + str(
            year_i)+"_082_Markup003.csv",header=None)
    else:
        print(year_i)
```

```
        df_trans_marg = pd.read_csv(dir + "/
↪21140000_annetest_AllCountries_199_T-Results_"+str(year_i)+"_082_Markup003.
↪csv", header=None)

# Add column and row names of the transportation margin tables:
    df_trans_marg.columns = df_cord_full.index.get_level_values(0)
    df_trans_marg['Ctry_Index'] =  df_cord_full.index.get_level_values(0)
    df_trans_marg['Ctry_Ent_Sec'] = df_cord_full.index.get_level_values(1)
    df_trans_marg.set_index(['Ctry_Index','Ctry_Ent_Sec'],inplace=True)

# From the concordance table (how the 14839 indusries aggregated to 26␣
↪sectors), get all industries in the hotels and restaurants industry:
    df_fs = df_cord_full[df_cord_full['Hotels and Restraurants']>0]
    df_fs = df_fs[['Hotels and Restraurants']]

    df_74_cou = pd.DataFrame(df_ctry['Ctry'])
    df_74_cou = df_74_cou.rename(columns={'Ctry': 'ROW'})

    df_fs_74 = df_fs[df_fs.index.get_level_values(0).str[:3].
↪isin(df_74_cou['ROW'])]
    df_fs_row = df_fs.T

    df_ag_food = df_cord_full[(df_cord_full.Agriculture>0) |(df_cord_full.
↪Fishing>0) | (df_cord_full['Food & Beverages']>0) | (df_cord_full.index.
↪get_level_values(0) =='ZZZ_Discrepancies')]
    df_ag_food = df_ag_food[['Agriculture','Fishing','Food & Beverages']]
    # only keep food service columns and related rows to reduce the data size:
    df_trans_marg = df_trans_marg[(df_trans_marg.index.get_level_values(0).
↪isin(df_ag_food.index.get_level_values(0)) )|(df_trans_marg.index.
↪get_level_values(0) == 'ZZZ_ROW') ]

    AFS_basic = T_.loc[:, (T_.columns.str.contains('A18'))]
    # 1. get basic prices of food services' demand:
    AFS_basic_A01 = AFS_basic[(AFS_basic.index.astype(str).str[4:7] == 'A01')]
    AFS_basic_A02 = AFS_basic[(AFS_basic.index.astype(str).str[4:7] == 'A02')]
    AFS_basic_A04 = AFS_basic[(AFS_basic.index.astype(str).str[4:7] == 'A04')]
    # 2. aggregate margins:
    # 1.aggregate rows to ag, fishing, and food & beverage:
    df_margin = df_trans_marg.copy()
    df_margin['Ctry'] = df_margin.index.get_level_values(0).str[:3]
    # df_margin.loc[~df_margin['Ctry'].isin(df_74_cou['ROW']), 'Ctry'] = 'ZZZ'
    df_margin.set_index('Ctry',append=True,drop=True,inplace=True)
    df_margin_A01 = df_margin.mul(df_ag_food['Agriculture'],axis=0)
    # don't have a detailed concordance tabel for the rest of the world. So not␣
↪estimating the transporation margins
    # for the rest of the world for now.
```

```
  df_margin_A01 = df_margin_A01.drop('ZZZ_ROW', axis=0)
  df_margin_A01 = df_margin_A01.groupby(df_margin_A01.index.
↪get_level_values(2)).sum()
  df_margin_A01.rename(index=lambda x: x + '_A01', inplace=True)
  # fishing:
  df_margin_A02 = df_margin.mul(df_ag_food['Fishing'],axis=0)
  df_margin_A02 = df_margin_A02.drop('ZZZ_ROW', axis=0)
  df_margin_A02 = df_margin_A02.groupby(df_margin_A02.index.
↪get_level_values(2)).sum()
  df_margin_A02.rename(index=lambda x: x + '_A02', inplace=True)
  # food & beverage:
  df_margin_A04 = df_margin.mul(df_ag_food['Food & Beverages'],axis=0)
  df_margin_A04 = df_margin_A04.drop('ZZZ_ROW', axis=0)
  df_margin_A04 = df_margin_A04.groupby(df_margin_A04.index.
↪get_level_values(2)).sum()
  df_margin_A04.rename(index=lambda x: x + '_A04', inplace=True)
  # 2. aggregate columns: The sum_col function will identify the food service␣
↪related columns,
  # and aggregate margins by country.

  df_trans_A01_ctry_colSum = sum_col(df_margin_A01, df_fs)
  df_trans_A02_ctry_colSum = sum_col(df_margin_A02, df_fs)
  df_trans_A04_ctry_colSum = sum_col(df_margin_A04, df_fs)

   # get margin ratios
  AFS_basic_A01_temp, trans_A01 = get_comm_rows_cols(AFS_basic_A01,␣
↪df_trans_A01_ctry_colSum)
  A01_trans_ratio = trans_A01.div(AFS_basic_A01_temp)

  columns_with_infA01 = A01_trans_ratio.columns[A01_trans_ratio.isin([np.inf,␣
↪-np.inf]).any()]
  A01_trans_ratio = A01_trans_ratio.replace([np.inf, -np.inf], 0)

  AFS_basic_A02_temp, trans_A02 = get_comm_rows_cols(AFS_basic_A02,␣
↪df_trans_A02_ctry_colSum)
  A02_trans_ratio = trans_A02.div(AFS_basic_A02_temp)
  columns_with_infA02 = A02_trans_ratio.columns[A02_trans_ratio.isin([np.inf,␣
↪-np.inf]).any()]
  A02_trans_ratio = A02_trans_ratio.replace([np.inf, -np.inf], 0)

  AFS_basic_A04_temp, trans_A04 = get_comm_rows_cols(AFS_basic_A04,␣
↪df_trans_A04_ctry_colSum)
  A04_trans_ratio = trans_A04.div(AFS_basic_A04_temp)
  columns_with_infA04 = A04_trans_ratio.columns[A04_trans_ratio.isin([np.inf,␣
↪-np.inf]).any()]
  A04_trans_ratio = A04_trans_ratio.replace([np.inf, -np.inf], 0)
```

```
    A01_trans_ratioL = Margin_win(A01_trans_ratio)
    A02_trans_ratioL = Margin_win(A02_trans_ratio)
    A04_trans_ratioL = Margin_win(A04_trans_ratio)

    A01_trans_ratioL['Ctry'] = A01_trans_ratioL['ROW'] + "_A01"
    A02_trans_ratioL['Ctry'] = A02_trans_ratioL['ROW'] + "_A02"
    A04_trans_ratioL['Ctry'] = A04_trans_ratioL['ROW'] + "_A04"
    trans_ratioL = pd.concat([A01_trans_ratioL, A02_trans_ratioL], axis=0,␣
↪ignore_index=True)
    trans_ratioL = pd.concat([trans_ratioL, A04_trans_ratioL], axis=0,␣
↪ignore_index=True)
    trans_ratioL['ROW'] = trans_ratioL['Ctry']
    trans_ratioL = trans_ratioL[['ROW','COL','Trans_ratio']]

    y_xh = X_.loc[:, X_.columns.str.endswith('_XH')]
    X_filtered_i = y_xh.copy()
    X_filtered_i = X_filtered_i.rename_axis('ROW').reset_index()
    X_filtered_iA1T4 = pd.melt(X_filtered_i, id_vars = 'ROW',␣
↪var_name='COL',value_name='XH')
    # Keep only rows where 'ROW' ends with 'A01', 'A02', or 'A04'
    X_filtered_iA1T4 = X_filtered_iA1T4[X_filtered_iA1T4['ROW'].str.
↪endswith(('A01', 'A02', 'A04'))]
    # Remove '_XH' from the 'COL' column
    X_filtered_iA1T4['COL'] = X_filtered_iA1T4['COL'].str.replace('_XH', '',␣
↪regex=False)
    X_filtered_iA1T4_trans = pd.merge(trans_ratioL, X_filtered_iA1T4,␣
↪on=['ROW','COL'])
    X_filtered_iA1T4_trans['Value'] = X_filtered_iA1T4_trans['Trans_ratio'] *␣
↪X_filtered_iA1T4_trans['XH']
    X_filtered_iA1T4_trans = X_filtered_iA1T4_trans[['ROW','COL','Value']]
    X_filtered_iA1T4_trans['Trans'] = X_filtered_iA1T4_trans['COL'] +'_A19'
    # X_filtered_iA1T4_trans[X_filtered_iA1T4_trans.COL=='ABW']
      # ==================Sep 23, 2024: Add final demand to calculate the margin␣
↪values: =========
    # X_filtered_iA1T4_trans['ROW'] = X_filtered_iA1T4_trans['COL']+'_A19'
    # X_filtered_iA1T4_trans.sum()
    # X_filtered_iA1T4_trans = X_filtered_iA1T4_trans.groupby('ROW')['Value'].
↪sum().reset_index()
    X_filtered_iA1T4_trans['Trans_Provider'] = X_filtered_iA1T4_trans['Trans']
    X_filtered_iA1T4_trans = X_filtered_iA1T4_trans[['Trans_Provider',␣
↪'ROW','COL','Value']]
    X_filtered_iA1T4_trans = X_filtered_iA1T4_trans.rename(columns = {'COL':
↪'Importer/Domestic','ROW':'COL'})
    X_filtered_iA1T4_trans = X_filtered_iA1T4_trans.rename(columns = {'Value':
↪'Margin_Tans'})
```

```
    X_filtered_iA1T4_trans.to_excel(OutputMargin+f'/transportation{year_i}.
↪xlsx',index=False)
```

### 2.3 Develop the trade margin:

[ ]:
```
for year_i in range(1993, 2022):
    # year_i = 1993
    print(year_i)
    df_out = pd.read_csv(OutputDir+'/Export/Exp_Agg_A27'+str(year_i)+'.
↪csv',sep='|', index_col='ROW')

    X_col = [col for col in df_out.columns if col[4:5] == 'X' ]
    T_ = df_out.drop(X_col, axis=1)
    l_list = ['LH01', 'LG01', 'LG02', 'LK01', 'LK02', 'LK03']
    T_ = T_[~T_.index.isin(l_list)]
    L_ = df_out[df_out.index.isin(l_list)]
    L_ = L_.drop(X_col, axis=1)
    X_ = df_out[X_col]
    X_ = X_[~X_.index.isin(l_list)]

    Ttemp = T_.copy()
    Ttemp = Ttemp.reset_index()
    Ttemp = Ttemp.melt(id_vars=['ROW'], var_name='COL',value_name='Value')
    negative_rows = Ttemp[Ttemp['Value'] < 0].copy()
    negative_rows_mapping = negative_rows.rename(columns={'ROW': 'COL', 'COL':␣
↪'ROW'})
    merged_df = pd.merge(Ttemp, negative_rows_mapping[['ROW', 'COL', 'Value']],␣
↪on=['ROW', 'COL'], how='left', suffixes=('', '_to_add'))
    Ttemp['Value'] = merged_df.apply(lambda row: row['Value'] + (-1) *␣
↪row['Value_to_add'] if pd.notnull(row['Value_to_add']) else row['Value'],␣
↪axis=1)
    Ttemp.loc[negative_rows.index, 'Value'] = 0
    Ttemp = Ttemp.pivot(index='ROW', columns='COL', values='Value')
    Ttemp = Ttemp.rename_axis('ROW')

    T_ = Ttemp.copy()
    T25 = A01T02(T_)
    L25 = A01T02(L_)
    X25 = A01T02(X_)


    df_trans_marg = pd.read_csv(OutputDir + '/TradeMargin' +'/' +␣
↪str(year_i)+"_TradeMargin.csv", header=None)

    df_trans_marg.columns = df_cord_full.index.get_level_values(0)
    df_trans_marg['Ctry_Index'] =  df_cord_full.index.get_level_values(0)
    df_trans_marg['Ctry_Ent_Sec'] = df_cord_full.index.get_level_values(1)
```

```
    df_trans_marg.set_index(['Ctry_Index','Ctry_Ent_Sec'],inplace=True)

    df_fs = df_cord_full[df_cord_full['Hotels and Restraurants']>0]
    df_fs = df_fs[['Hotels and Restraurants']]

    df_74_cou = pd.DataFrame(df_ctry['Ctry'])
    df_74_cou = df_74_cou.rename(columns={'Ctry': 'ROW'})
    # for ILO data:
    df_fs_74 = df_fs[df_fs.index.get_level_values(0).str[:3].
↪isin(df_74_cou['ROW'])]
    df_fs_row = df_fs.T

    df_ag_food = df_cord_full[(df_cord_full.Agriculture>0) |(df_cord_full.
↪Fishing>0) | (df_cord_full['Food & Beverages']>0) | (df_cord_full.index.
↪get_level_values(0) =='ZZZ_Discrepancies')]
    df_ag_food = df_ag_food[['Agriculture','Fishing','Food & Beverages']]
    # only keep food service columns and related rows to reduce the data size:
    df_trans_marg = df_trans_marg[(df_trans_marg.index.get_level_values(0).
↪isin(df_ag_food.index.get_level_values(0)) )|(df_trans_marg.index.
↪get_level_values(0) == 'ZZZ_ROW') ]


    AFS_basic = T_.loc[:, (T_.columns.str.contains('A18'))]
    # 1. get basic prices of food services' demand:
    AFS_basic_A01 = AFS_basic[(AFS_basic.index.astype(str).str[4:7] == 'A01')]
    AFS_basic_A02 = AFS_basic[(AFS_basic.index.astype(str).str[4:7] == 'A02')]
    AFS_basic_A04 = AFS_basic[(AFS_basic.index.astype(str).str[4:7] == 'A04')]
    # 2. aggregate margins:
    # 1.aggregate rows to ag, fishing, and food & beverage:
    # df_trans_A01,df_trans_A02, df_trans_A04 = get_fs_margin(df_trans)
    df_margin = df_trans_marg.copy()
    df_margin['Ctry'] = df_margin.index.get_level_values(0).str[:3]
    # df_margin.loc[~df_margin['Ctry'].isin(df_74_cou['ROW']), 'Ctry'] = 'ZZZ'
    df_margin.set_index('Ctry',append=True,drop=True,inplace=True)
    df_margin_A01 = df_margin.mul(df_ag_food['Agriculture'],axis=0)
    # don't have a detailed concordance tabel for the rest of the world. So not␣
↪estimating the transporation margins
    # for the rest of the world for now.
    df_margin_A01 = df_margin_A01.drop('ZZZ_ROW', axis=0)
    df_margin_A01 = df_margin_A01.groupby(df_margin_A01.index.
↪get_level_values(2)).sum()
    df_margin_A01.rename(index=lambda x: x + '_A01', inplace=True)
    # fishing:
    df_margin_A02 = df_margin.mul(df_ag_food['Fishing'],axis=0)
    df_margin_A02 = df_margin_A02.drop('ZZZ_ROW', axis=0)
    df_margin_A02 = df_margin_A02.groupby(df_margin_A02.index.
↪get_level_values(2)).sum()
```

```
    df_margin_A02.rename(index=lambda x: x + '_A02', inplace=True)
    # food & beverage:
    df_margin_A04 = df_margin.mul(df_ag_food['Food & Beverages'],axis=0)
    df_margin_A04 = df_margin_A04.drop('ZZZ_ROW', axis=0)
    df_margin_A04 = df_margin_A04.groupby(df_margin_A04.index.
↪get_level_values(2)).sum()
    df_margin_A04.rename(index=lambda x: x + '_A04', inplace=True)
    # 2. aggregate columns: The sum_col function will identify the food service␣
↪related columns,
    # and aggregate margins by country.

    df_trans_A01_ctry_colSum = sum_col(df_margin_A01, df_fs)
    df_trans_A02_ctry_colSum = sum_col(df_margin_A02, df_fs)
    df_trans_A04_ctry_colSum = sum_col(df_margin_A04, df_fs)


     # get margin ratios
    AFS_basic_A01_temp, trans_A01 = get_comm_rows_cols(AFS_basic_A01,␣
↪df_trans_A01_ctry_colSum)
    A01_trans_ratio = trans_A01.div(AFS_basic_A01_temp)

    columns_with_infA01 = A01_trans_ratio.columns[A01_trans_ratio.isin([np.inf,␣
↪-np.inf]).any()]
    A01_trans_ratio = A01_trans_ratio.replace([np.inf, -np.inf], 0)

    AFS_basic_A02_temp, trans_A02 = get_comm_rows_cols(AFS_basic_A02,␣
↪df_trans_A02_ctry_colSum)
    A02_trans_ratio = trans_A02.div(AFS_basic_A02_temp)
    columns_with_infA02 = A02_trans_ratio.columns[A02_trans_ratio.isin([np.inf,␣
↪-np.inf]).any()]
    A02_trans_ratio = A02_trans_ratio.replace([np.inf, -np.inf], 0)

    AFS_basic_A04_temp, trans_A04 = get_comm_rows_cols(AFS_basic_A04,␣
↪df_trans_A04_ctry_colSum)
    A04_trans_ratio = trans_A04.div(AFS_basic_A04_temp)
    columns_with_infA04 = A04_trans_ratio.columns[A04_trans_ratio.isin([np.inf,␣
↪-np.inf]).any()]
    A04_trans_ratio = A04_trans_ratio.replace([np.inf, -np.inf], 0)

    A01_trans_ratioL = Margin_win(A01_trans_ratio)
    A02_trans_ratioL = Margin_win(A02_trans_ratio)
    A04_trans_ratioL = Margin_win(A04_trans_ratio)

    A01_trans_ratioL['Ctry'] = A01_trans_ratioL['ROW'] + "_A01"
    A02_trans_ratioL['Ctry'] = A02_trans_ratioL['ROW'] + "_A02"
    A04_trans_ratioL['Ctry'] = A04_trans_ratioL['ROW'] + "_A04"
```

```
   trans_ratioL = pd.concat([A01_trans_ratioL, A02_trans_ratioL], axis=0,␣
↪ignore_index=True)
   trans_ratioL = pd.concat([trans_ratioL, A04_trans_ratioL], axis=0,␣
↪ignore_index=True)
   trans_ratioL['ROW'] = trans_ratioL['Ctry']
   trans_ratioL = trans_ratioL[['ROW','COL','Trans_ratio']]
   y_xh = X_.loc[:, X_.columns.str.endswith('_XH')]
   X_filtered_i = y_xh.copy()
   X_filtered_i = X_filtered_i.rename_axis('ROW').reset_index()
   X_filtered_iA1T4 = pd.melt(X_filtered_i, id_vars = 'ROW',␣
↪var_name='COL',value_name='XH')
   # Keep only rows where 'ROW' ends with 'A01', 'A02', or 'A04'
   X_filtered_iA1T4 = X_filtered_iA1T4[X_filtered_iA1T4['ROW'].str.
↪endswith(('A01', 'A02', 'A04'))]
   # Remove '_XH' from the 'COL' column
   X_filtered_iA1T4['COL'] = X_filtered_iA1T4['COL'].str.replace('_XH', '',␣
↪regex=False)
   X_filtered_iA1T4_trans = pd.merge(trans_ratioL, X_filtered_iA1T4,␣
↪on=['ROW','COL'])
   X_filtered_iA1T4_trans['Value'] = X_filtered_iA1T4_trans['Trans_ratio'] *␣
↪X_filtered_iA1T4_trans['XH']
   X_filtered_iA1T4_trans = X_filtered_iA1T4_trans[['ROW','COL','Value']]
   X_filtered_iA1T4_trans['Trade'] = X_filtered_iA1T4_trans['COL'] +'_A19'

     # =================Add final demand to calculate the margin values:␣
↪=========

   X_filtered_iA1T4_trans['Trade_Provider'] = X_filtered_iA1T4_trans['Trade']
   X_filtered_iA1T4_trans = X_filtered_iA1T4_trans[['Trans_Provider',␣
↪'ROW','COL','Value']]
   X_filtered_iA1T4_trans = X_filtered_iA1T4_trans.rename(columns = {'COL':
↪'Importer/Domestic','ROW':'COL'})
   X_filtered_iA1T4_trans['Trade_Provider'] = X_filtered_iA1T4_trans['Importer/
↪Domestic'] + '_A16_A17'
   X_filtered_iA1T4_trans = X_filtered_iA1T4_trans.rename(columns = {'Value':
↪'Margin_Trade'})
   X_filtered_iA1T4_trans.to_excel(OutputMargin+f'/trade{year_i}.
↪xlsx',index=False)
   X_filtered_iA1T4_trans.sum()
```

### 2.4 Develop wholesale and retail share:

To disaggregate trade margins between wholesale and retail industries, household demand for wholesale and retail services is used to allocate the total trade margin between these two sectors.

[ ]:
```
Dir_shr = r"D:\...\TradeShr"
for year_i in range(1993, 2022):
    # year_i = 1993
    print(year_i)
    df_out = pd.read_csv(OutputDir+'/Export/Exp_Agg_A27'+str(year_i)+'.
 ↪csv',sep='|', index_col='ROW') # df_out is the balanced EORA 26 annual table

    X_col = [col for col in df_out.columns if col[4:5] == 'X' ]
    l_list = ['LH01', 'LG01', 'LG02', 'LK01', 'LK02', 'LK03']

    X_ = df_out[X_col]
    X_ = X_[~X_.index.isin(l_list)]
    y_xh = X_.loc[:, X_.columns.str.endswith('_XH')]
    X_filtered_i = y_xh.rename_axis('ROW').reset_index()
    X_filtered_iA1T4 = pd.melt(X_filtered_i, id_vars = 'ROW',␣
 ↪var_name='COL',value_name='XH')
    X_filtered_iA1T4_filtered = X_filtered_iA1T4[X_filtered_iA1T4['ROW'].str[:
 ↪3] == X_filtered_iA1T4['COL'].str[:3]]
    X_filtered_iA1T4_filtered =␣
 ↪X_filtered_iA1T4_filtered[X_filtered_iA1T4_filtered['ROW'].str[-2:].
 ↪isin(['16', '17'])]

    X_filtered_iA1T4_filtered['Country'] = X_filtered_iA1T4_filtered['ROW'].
 ↪str[:3]
    X_filtered_iA1T4_filtered['Industry'] = X_filtered_iA1T4_filtered['ROW'].
 ↪str[-3:]

    # Step 2: Create 'A16_XH' and 'A17_XH' columns based on 'Industry'
    # If Industry is 'A16', assign 'XH' value to 'A16_XH'. Otherwise, it will␣
 ↪be 0.
    X_filtered_iA1T4_filtered['A16_XH'] = X_filtered_iA1T4_filtered.apply(
        lambda row: row['XH'] if row['Industry'] == 'A16' else 0, axis=1)

    # If Industry is 'A17', assign 'XH' value to 'A17_XH'. Otherwise, it will␣
 ↪be 0.
    X_filtered_iA1T4_filtered['A17_XH'] = X_filtered_iA1T4_filtered.apply(
        lambda row: row['XH'] if row['Industry'] == 'A17' else 0, axis=1)

    # Step 3: Group by 'Country' and sum 'A16_XH' and 'A17_XH'
    country_grouped = X_filtered_iA1T4_filtered.groupby('Country').agg(
        A16_XH=('A16_XH', 'sum'),
        A17_XH=('A17_XH', 'sum')
    ).reset_index()

    # Step 4: Calculate total 'XH' for each country as the sum of 'A16_XH' and␣
 ↪'A17_XH'
```

```
    country_grouped['XH'] = country_grouped['A16_XH'] +␣
↪country_grouped['A17_XH']

    # Step 5: Calculate 'A16_share' and 'A17_share'
    # Shares are the proportion of 'A16_XH' and 'A17_XH' relative to total 'XH'
    country_grouped['A16_share'] = country_grouped['A16_XH'] /␣
↪country_grouped['XH']
    country_grouped['A17_share'] = country_grouped['A17_XH'] /␣
↪country_grouped['XH']
    country_grouped = country_grouped[['Country','A16_share','A17_share' ]]
    country_grouped.to_excel(f"{Dir_shr}/{year_i}_Shr.xlsx", index=False)
```

# 3 Appendix-Country List

| CTRY | COUNTRY |
|---|---|
| AFG | Afghanistan |
| ALB | Albania |
| DZA | Algeria |
| AND | Andorra |
| AGO | Angola |
| ATG | Antigua |
| ARG | Argentina |
| ARM | Armenia |
| ABW | Aruba |
| AUS | Australia |
| AUT | Austria |
| AZE | Azerbaijan |
| BHS | Bahamas |
| BHR | Bahrain |
| BGD | Bangladesh |
| BRB | Barbados |
| BLR | Belarus |
| BEL | Belgium |
| BLZ | Belize |
| BEN | Benin |
| BMU | Bermuda |
| BTN | Bhutan |
| BOL | Bolivia |
| BIH | Bosnia and Herzegovina |
| BWA | Botswana |
| BRA | Brazil |
| VGB | British Virgin Islands |
| BRN | Brunei |
| BGR | Bulgaria |
| BFA | Burkina Faso |
| BDI | Burundi |

| CTRY | COUNTRY |
|---|---|
| KHM | Cambodia |
| CMR | Cameroon |
| CAN | Canada |
| CPV | Cape Verde |
| CYM | Cayman Islands |
| CAF | Central African Republic |
| TCD | Chad |
| CHL | Chile |
| CHN | China |
| COL | Colombia |
| COG | Congo |
| CRI | Costa Rica |
| HRV | Croatia |
| CUB | Cuba |
| CYP | Cyprus |
| CZE | Czech Republic |
| CIV | Cote dIvoire |
| PRK | North Korea |
| COD | DR Congo |
| DNK | Denmark |
| DJI | Djibouti |
| DOM | Dominican Republic |
| ECU | Ecuador |
| EGY | Egypt |
| SLV | El Salvador |
| ERI | Eritrea |
| EST | Estonia |
| ETH | Ethiopia |
| FJI | Fiji |
| FIN | Finland |
| FRA | France |
| PYF | French Polynesia |
| GAB | Gabon |
| GMB | Gambia |
| GEO | Georgia |
| DEU | Germany |
| GHA | Ghana |
| GRC | Greece |
| GRL | Greenland |
| GTM | Guatemala |
| GIN | Guinea |
| GUY | Guyana |
| HTI | Haiti |
| HND | Honduras |
| HKG | Hong Kong |
| HUN | Hungary |

| CTRY | COUNTRY |
|---|---|
| ISL | Iceland |
| IND | India |
| IDN | Indonesia |
| IRN | Iran |
| IRQ | Iraq |
| IRL | Ireland |
| ISR | Israel |
| ITA | Italy |
| JAM | Jamaica |
| JPN | Japan |
| JOR | Jordan |
| KAZ | Kazakhstan |
| KEN | Kenya |
| KWT | Kuwait |
| KGZ | Kyrgyzstan |
| LAO | Laos |
| LVA | Latvia |
| LBN | Lebanon |
| LSO | Lesotho |
| LBR | Liberia |
| LBY | Libya |
| LIE | Liechtenstein |
| LTU | Lithuania |
| LUX | Luxembourg |
| MAC | Macao SAR |
| MDG | Madagascar |
| MWI | Malawi |
| MYS | Malaysia |
| MDV | Maldives |
| MLI | Mali |
| MLT | Malta |
| MRT | Mauritania |
| MUS | Mauritius |
| MEX | Mexico |
| MCO | Monaco |
| MNG | Mongolia |
| MNE | Montenegro |
| MAR | Morocco |
| MOZ | Mozambique |
| MMR | Myanmar |
| NAM | Namibia |
| NPL | Nepal |
| NLD | Netherlands |
| ANT | Netherlands Antilles |
| NCL | New Caledonia |
| NZL | New Zealand |

| CTRY | COUNTRY |
|---|---|
| NIC | Nicaragua |
| NER | Niger |
| NGA | Nigeria |
| NOR | Norway |
| PSE | Gaza Strip |
| OMN | Oman |
| PAK | Pakistan |
| PAN | Panama |
| PNG | Papua New Guinea |
| PRY | Paraguay |
| PER | Peru |
| PHL | Philippines |
| POL | Poland |
| PRT | Portugal |
| QAT | Qatar |
| KOR | South Korea |
| MDA | Moldova |
| ROU | Romania |
| RUS | Russia |
| RWA | Rwanda |
| WSM | Samoa |
| SMR | San Marino |
| STP | Sao Tome and Principe |
| SAU | Saudi Arabia |
| SEN | Senegal |
| SRB | Serbia |
| SYC | Seychelles |
| SLE | Sierra Leone |
| SGP | Singapore |
| SVK | Slovakia |
| SVN | Slovenia |
| SOM | Somalia |
| ZAF | South Africa |
| SDS | South Sudan |
| ESP | Spain |
| LKA | Sri Lanka |
| SUD | Sudan |
| SUR | Suriname |
| SWZ | Swaziland |
| SWE | Sweden |
| CHE | Switzerland |
| SYR | Syria |
| TWN | Taiwan |
| TJK | Tajikistan |
| THA | Thailand |
| MKD | TFYR Macedonia |

| CTRY | COUNTRY |
|---|---|
| TGO | Togo |
| TTO | Trinidad and Tobago |
| TUN | Tunisia |
| TUR | Turkey |
| TKM | Turkmenistan |
| USR | Former USSR |
| UGA | Uganda |
| UKR | Ukraine |
| ARE | UAE |
| GBR | UK |
| TZA | Tanzania |
| USA | USA |
| URY | Uruguay |
| UZB | Uzbekistan |
| VUT | Vanuatu |
| VEN | Venezuela |
| VNM | Viet Nam |
| YEM | Yemen |
| ZMB | Zambia |
| ZWE | Zimbabwe |